

JAVASCRIPT TEMELLERİ

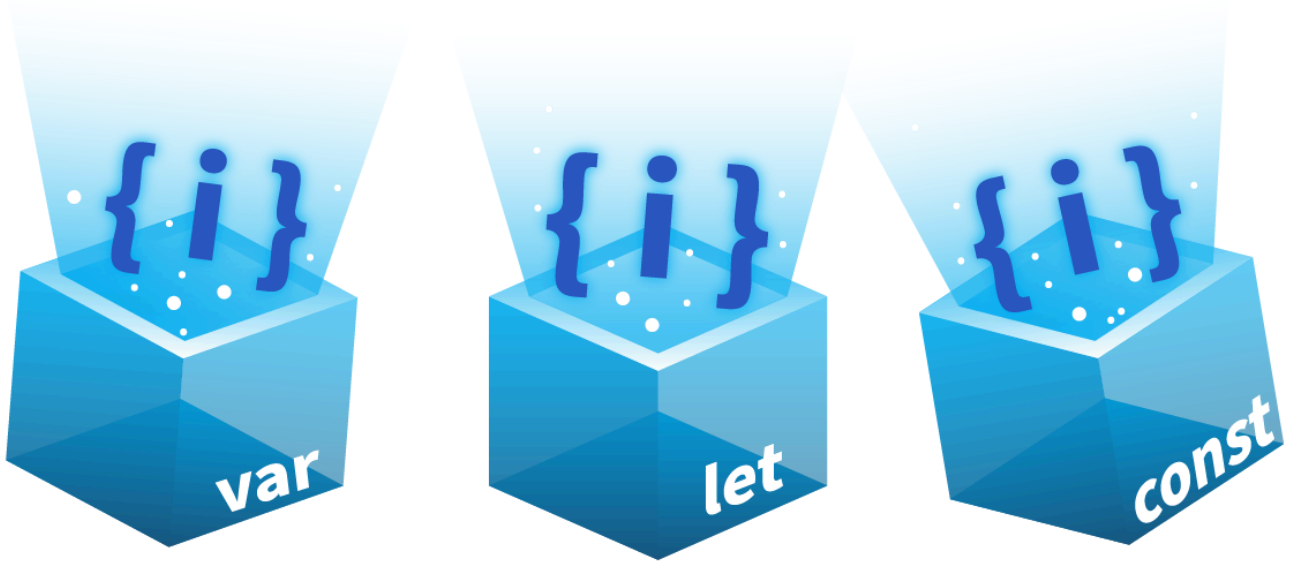
[Kaynak](#)

İÇERİK

1. JavaScript Temelleri ve Web Çalışma Mantığı: Essential Concepts
2. JavaScript Genel Bakış ve İlk Kod Deneyimi
3. JavaScript'in HTML'e Entegrasyonu: İçsel ve Dışsal Yöntemler
4. Diyalog Fonksiyonlarının Kötüye Kullanımı (Abusing Dialogue Functions)
5. Kontrol Akış Yapılarını Atlama (Bypassing Control Flow Statements)
6. Minified ve Obfuscated (Karmaşılaştırılmış) Dosyaları İnceleme
7. JavaScript Güvenliği: En İyi Uygulamalar (Best Practices)

JavaScript Temelleri ve Web Çalışma Mantığı: Essential Concepts

Bu notlar, bir siber güvenlik araştırmacısının web tabanlı saldırıları (XSS, CSRF, JS Injection vb.) anlayabilmesi için bilmesi gereken temel **JavaScript (JS)** yapı taşlarını ve istemci-sunucu arasındaki veri trafiğinin mantığını kapsar. Kodun nasıl çalıştığını anlamadan, o kodu nasıl manipüle edeceğimizi bilemeyiz.



Değişkenler (Variables)

JavaScript'te değişkenler, verileri içinde barındıran **konteynırlar** gibidir. Gerçek hayatta bir kutunun içine bir şey koyduğumuzda, daha sonra onu kolayca bulabilmek için kutunun üzerine bir etiket yapıştırırız. Yazılımda da bu etikete **değişken adı** diyoruz. JS dünyasında bir değişken tanımlamanın üç farklı yolu vardır ve bunların kapsamı (**scope**), güvenlik testlerinde değişkenin nereden erişilebilir olduğunu belirlediği için kritiktir:

- **var**: Fonksiyon kapsamlıdır (**function-scoped**). Yani tanımlandığı fonksiyonun her yerinden erişilebilir. Eski bir yöntemdir ve bazen beklenmedik kapsama sorunlarına yol açabilir.
- **let**: Blok kapsamlıdır (**block-scoped**). Sadece tanımlandığı süslü parantez { } içerisinde geçerlidir. Günümüzde tercih edilen yöntemdir.
- **const**: Sabitleri ifade eder. Değeri bir kez atandıktan sonra değiştirilemez. Bu da blok kapsamlıdır.

Veri Tipleri (Data Types)

JS'te bir değişkenin ne tür bir veri tuttuğunu bilmek, o veriyle ne tür işlemler yapabileceğimizi belirler. Siber güvenlikte, özellikle "type juggling" veya beklenmeyen veri tipi girişleriyle (örneğin bir sayı beklenen yere bir obje gönderilmesi) sistemleri manipüle etmek mümkündür.

- **String**: Metinsel ifadelerdir. "Merhaba" gibi.
- **Number**: Tam sayılar veya ondalıklı sayılar.
- **Boolean**: Mantıksal değerler; sadece `true` (doğru) veya `false` (yanlış) değerini alabilir.
- **Null**: Değişkenin kasıtlı olarak boş bırakıldığını belirtir.
- **Undefined**: Değişkene henüz bir değer atanmadığını gösterir.
- **Object**: Diziler (**Arrays**) veya daha karmaşık veri yapılarını tutmak için kullanılır.

Fonksiyonlar (Functions)

Fonksiyonlar, belirli bir görevi yerine getirmek için tasarlanmış kod bloklarıdır. Sürekli tekrar eden işlemleri her seferinde baştan yazmak yerine, bir fonksiyon oluşturup ihtiyaç duyduğumuzda onu çağırırız (**invoke**).

Örneğin, bir web sayfasında 100 öğrencinin sınav sonucunu yazdırmak istiyorsak, her öğrenci için ayrı ayrı kod yazmayız. Bunun yerine öğrenci numarasını (**argument**) kabul eden tek bir fonksiyon yazarız:

JavaScript

```
<script>
  // PrintResult adında bir fonksiyon tanımlıyoruz.
  // Parantez içindeki 'rollNum', dışarıdan alacağı veriyi temsil eder.
  function PrintResult(rollNum) {
    // alert() fonksiyonu tarayıcıda bir uyarı penceresi açar.
    // Metinleri birleştirmek için + operatörü kullanılır.
    alert("Username with roll number " + rollNum + " has passed the
exam");
  }

  // rollNumbers adında sabit bir dizi (array) tanımlıyoruz.
```

```
const rollNumbers = [101, 102, 103];

// Döngü kısmında bu fonksiyonun nasıl çağrıldığını göreceğiz.
</script>
```

Siber güvenlik perspektifinde, bu fonksiyonların içine sızdırılan zararlı girdiler (örneğin `rollNum` yerine bir script kodu yazılması) **XSS (Cross-Site Scripting)** açıklarına neden olur.

Döngüler (Loops)

Döngüler, belirli bir koşul **doğru (true)** olduğu sürece bir kod bloğunu tekrar tekrar çalıştırmamıza olanak tanır. JavaScript'te en sık kullanılan döngüler **for**, **while** ve **do...while** yapılarıdır.

Yukarıdaki öğrenci listesi örneğinde, fonksiyonu tek tek elle çağırmak yerine bir `for` döngüsü ile otomatize edebiliriz:

JavaScript

```
// Elimizde 3 adet veri var: 101, 102 ve 103
const rollNumbers = [101, 102, 103];

// for döngüsü: i değişkeni 0'dan başlar, 100'den küçük olduğu sürece devam eder.
// i++ ifadesi her adımda i değerini 1 artırır.
for (let i = 0; i < 100; i++) {
    // PrintResult fonksiyonu 100 kez çağrılıyor.
    PrintResult(rollNumbers[i]);
}
```

Kritik Detay: Yukarıdaki kodda dizi sadece 3 elemanlıdır (`index 0, 1, 2`). Ancak döngü 100'e kadar gitmektedir. `i` değeri 2'den büyük olduğunda `rollNumbers[i]` ifadesi **undefined** dönecektir. Bu durum, programın beklenmedik çıktılar üretmesine veya hatalara yol açmasına sebep olabilir (Logic Error).

İstek-Yanıt Döngüsü (Request-Response Cycle)

Web güvenliğinin temelini bu döngü oluşturur. Her şey istemci (**Client** - yani bizim tarayıcımız) ve sunucu (**Server**) arasındaki iletişimle başlar:

1. **Request (İstek):** Kullanıcı bir URL yazdığında veya bir butona tıkladığında, tarayıcı sunucuya bir istek gönderir. Bu istekte hangi sayfanın istendiği, kullanıcı bilgileri (cookies) ve varsa form verileri yer alır.
2. **Response (Yanıt):** Sunucu isteği alır, işler (veritabanına bakar, kodları çalıştırır) ve tarayıcıya bir yanıt döner. Bu yanıt genellikle bir HTML sayfası, bir resim veya JSON formatında bir veridir.

Siber saldırıların büyük çoğunluğu bu döngünün arasına girmek (**Man-in-the-Middle**), isteği manipüle etmek veya sunucunun verdiği yanıtı tarayıcıda yanlış yorumlatmak üzerine kuruludur.

JavaScript Genel Bakış ve İlk Kod Deneyimi

Bu bölüm, JavaScript'in çalışma mantığını kavramak ve tarayıcı üzerinde ilk kodumuzu koşturmak (execute) üzerine odaklanıyor. Bir siber güvenlikçi için en önemli nokta şudur:

JavaScript yorumlanan (interpreted) bir dildir. Yani kod, sunucuda derlenip bize gelmez; ham haliyle tarayıcımıza iner ve bizim bilgisayarımızda çalıştırılır. Bu da bize kodu inceleme, durdurma ve manipüle etme şansı tanır.

JavaScript'in Doğası: Yorumlanan Dil

JavaScript, **interpreted** bir dildir. Bu, kodun önceden bir derleyici (compiler) tarafından makine diline çevrilmesine gerek olmadığı anlamına gelir. Tarayıcı (Chrome, Firefox vb.), kodu satır satır okur ve o an çalıştırır.

- Client-Side (İstemci Tarafı):** JS esas olarak kullanıcının tarayıcısında çalışır. Bu sayede HTML öğelerine doğrudan müdahale edebilir, sayfa yenilenmeden içeriği değiştirebiliriz.
- İnceleme Kolaylığı:** Kod bizim cihazımızda çalıştığı için, tarayıcı araçlarını kullanarak herhangi bir web sitesinin JS mantığını çözebiliriz.

Temel JS Yapı Taşları (Örnek Program)

Aşağıdaki basit "Hello World" ve kontrol yapısı örneği, JS'in nasıl görüldüğünü anlamamıza yardımcı olur:

JavaScript

```
// Konsola çıktı vermek için kullanılır, hata ayıklama (debugging) için
temel araçtır.
console.log("Hello, World!");

// Değişken Tanımlama: 'let' ile 'age' adında sayı (number) tipinde bir
değişken oluşturduk.
let age = 25;

// Kontrol Akış Yapısı (If-Else): Belirli bir koşula göre kodun yönünü
değiştirir.
if (age >= 18) {
  // Eğer yaş 18 veya daha büyükse burası çalışır.
  console.log("You are an adult.");
} else {
  // Koşul sağlanmazsa burası çalışır.
  console.log("You are a minor.");
}
```

```
// Fonksiyon Tanımlama: Tekrar kullanılabilir bir kod bloğu oluşturur.
function greet(name) {
    console.log("Hello, " + name + "!");
}

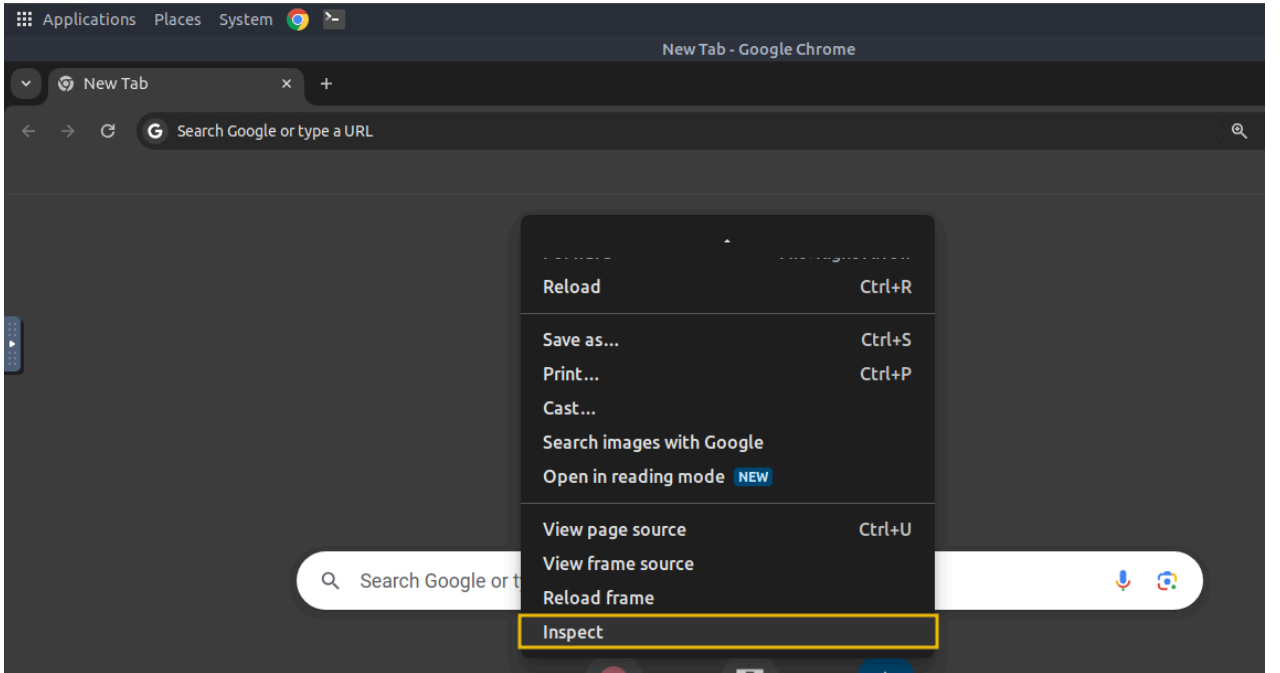
// Fonksiyonu Çağırma: 'Bob' argümanını göndererek fonksiyonu
çalıştırıyoruz.
greet("Bob");
```

İlk Programı Çalıştırma: Google Chrome Console

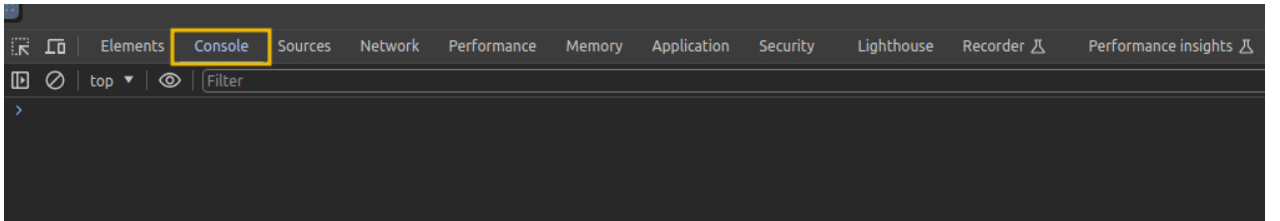
Ek bir yazılım yüklemeye gerek kalmadan, tarayıcının kendi araçlarını (DevTools) kullanarak kod yazabiliriz. Bu, siber güvenlik testlerinde (örneğin bir XSS payload'unu test ederken) en sık kullanacağımız alandır.

Adım Adım Uygulama:

1. **Konsolu Açma:** Tarayıcıda sağ tıklayıp **İncele (Inspect)** deyin veya **Ctrl + Shift + I** kısayolunu kullanın.



2. **Console Sekmesi:** Üstteki menüden **Console** sekmesine tıklayın. Burası doğrudan JS komutlarını kabul eden bir terminal gibidir.



3. **Kodu Çalıştırma:** Aşağıdaki toplama işlemini yapan kodu kopyalayıp konsola yapıştırın ve **Enter** tuşuna basın.

```
let x = 5;
let y = 10;
let result = x + y;
console.log("The result is: " + result);
```

Deneyilecek Kod Bloğu:

JavaScript

```
// Değişkenlere değer atıyoruz
let x = 5;
let y = 10;

// İki değişkeni toplayıp 'result' adlı yeni bir değişkene aktarıyoruz
let result = x + y;

// Sonucu konsol ekranına yazdırıyoruz
console.log("The result is: " + result);
```

Kodun Mantığı:

- burada `x` ve `y` sayı tutan **değişkenlerdir**.
- `x + y` bir **ifadedir (expression)**; iki sayıyı toplar.
- `console.log`, sonucu geliştirici ekranına (konsola) basan bir **fonksiyondur**.

Not: Konsola yazdığınız kodlar anlık olarak tarayıcı belleğinde çalışır. Sayfayı yenilediğinizde tanımladığınız değişkenler silinecektir.

JavaScript'in HTML'e Entegrasyonu: İçsel ve Dışsal Yöntemler

Bu bölüm, JavaScript kodunun bir web sayfasına nasıl dahil edildiğini ve bir siber güvenlik araştırmacısı (pentester) için bu ayrımın neden önemli olduğunu ele alıyor. JS, sayfada içerik oluşturmaktan ziyade HTML ve CSS ile etkileşime girerek sayfayı **dinamik** hale getirir.

1. Dahili JavaScript (Internal JavaScript)

Dahili JS, kodun doğrudan HTML dosyasının içine, `<script>` etiketleri arasına yazılmasıdır.

- **Nereye Yazılır?:** Genellikle `<head>` (sayfa yüklenmeden önce çalışması gerekenler için) veya `<body>` (sayfa öğeleriyle etkileşime girenler için) bölümlerine yerleştirilir.
- **Avantajı:** Küçük scriptler için hızlıdır; HTML ve JS arasındaki etkileşimi tek bir dosyada görmeyi sağlar.

Uygulama Örneği: `internal.html`

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>Internal JS</title>
</head>
<body>
  <h1>Addition of Two Numbers</h1>
  <p id="result"></p>

  <script>
    let x = 5;
    let y = 10;
    let result = x + y;

    // DOM Manipülasyonu: "result" ID'li elementi bul ve içeriğini
    değiştir.
    document.getElementById("result").innerHTML = "The result is: " +
    result;
  </script>
</body>
</html>
```

Kod Analizi:

- `document.getElementById("result")` : Tarayıcıya, ID'si "result" olan HTML etiketini (burada `<p>`) bulmasını söyler.
- `.innerHTML` : Seçilen bu etiketin içindeki metni veya HTML içeriğini hedef alır.
- Bu yöntemle JS, statik olan HTML sayfasına çalışma anında müdahale etmiş olur.

2. Harici JavaScript (External JavaScript)

Kodun `.js` uzantılı ayrı bir dosyada tutulması ve HTML'e bir bağlantı (link) aracılığıyla dahil edilmesidir. Büyük projelerde kodun düzenli kalması ve bakımı için bu yöntem standarttır.

Uygulama Örneği: `script.js` ve `external.html`

Adım 1: JS Dosyası (`script.js`)

Bu dosyada sadece saf JavaScript kodları bulunur, HTML etiketleri kullanılmaz.

JavaScript

```
let x = 5;
let y = 10;
```

```
let result = x + y;
document.getElementById("result").innerHTML = "The result is: " + result;
```

Adım 2: HTML Dosyası (external.html)

Burada JS kodlarını çağırmak için `src` (source) özniteliği kullanılır.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>External JS</title>
</head>
<body>
  <h1>Addition of Two Numbers</h1>
  <p id="result"></p>

  <script src="script.js"></script>
</body>
</html>
```

Kritik Fark: Tarayıcı bu sayfayı yüklerken `src="script.js"` kısmını gördüğünde, sunucuya gidip o dosyayı ayrıca indirir ve çalıştırır. Sonuç kullanıcı için aynıdır ancak dosya yapısı daha profesyoneldir.

3. Güvenlik ve Analiz: Dahili mi, Harici mi?

Bir web uygulamasına sızma testi (pen-testing) yaparken, ilk adımlardan biri **Kaynak Kodu Görüntüle (View Page Source)** diyerek JS'nin nasıl yüklendiğini incelemektir.

Analiz Tablosu

Görünüm	Türü	Güvenlik Notu
<code><script> ...kodlar... </script></code>	Dahili (Internal)	Sayfa kaynağında tüm mantık kabak gibi ortadadır. Analizi çok kolaydır.
<code><script src="dosya.js"> </script></code>	Harici (External)	Kod gizli değildir ancak incelemek için dosya yoluna (<code>src</code>) tıklayıp o dosyayı ayrıca açman gerekir.


```
1 <!DOCTYPE html>
2 <html>
3 <head>
4 <title>TryHackMe | Cyber Security Training</title>
5 <meta name="description" content="TryHackMe is a free online platform for learning cyber security, using hands-on exercises and labs, all through your browser!" />
6 <meta name="og:description" content="TryHackMe is a free online platform for learning cyber security, using hands-on exercises and labs, all through your browser!" />
7 <meta name="keywords" content="cyber,security,cyber security,cyber security training,coding,computer,bitcoin,hacking,hackers,hacks,hack,exploits,keylogger,learn,poc" />
8 <meta name="viewport" content="width=device-width,initial-scale=1.0" />
9
10 <meta property="og:site_name" content="TryHackMe">
11 <meta property="og:title" content="TryHackMe | Cyber Security Training">
12 <meta property="og:image" content="https://tryhackme.com/img/meta/default.png">
13 <meta property="og:url" content="https://tryhackme.com">
14 <meta name="twitter:image" content="https://tryhackme.com/img/meta/default.png">
15 <meta property="og:description" content="An online platform for learning and teaching cyber security, all through your browser.">
16 <meta charset="utf-8">
17 <script src="https://assets.tryhackme.com/js/jquery.min.js?v=3.5.1"></script>
18 <script src="https://assets.tryhackme.com/js/popper.min.js"></script>
19 <link rel="stylesheet" href="https://assets.tryhackme.com/css/bootstrap.min.css">
20 <script src="https://assets.tryhackme.com/js/bootstrap.min.js"></script>
21 <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/3.7.2/animate.min.css">
22 <link rel="stylesheet" href="https://pro.fontawesome.com/releases/v5.12.0/css/all.css" integrity="sha384-ekOryaXPbCpHQixwSwVvQ0+1VrStoPjQ54sh1YhR8HzQgig1v5fas6YgQqLkz" crossorigin="anonymous">
23 <link rel="icon" type="image/png" href="https://assets.tryhackme.com/img/favicon.png" />
24 <link rel="stylesheet" media="screen" href="https://assets.tryhackme.com/css/general-style.css?v=2.17">
25 <script src="https://assets.tryhackme.com/js/script.js?v=3.12"></script>
26 <script src="https://assets.tryhackme.com/js/validation.js"></script>
27 <script type="text/javascript">
```

Nasıl Kontrol Edilir?

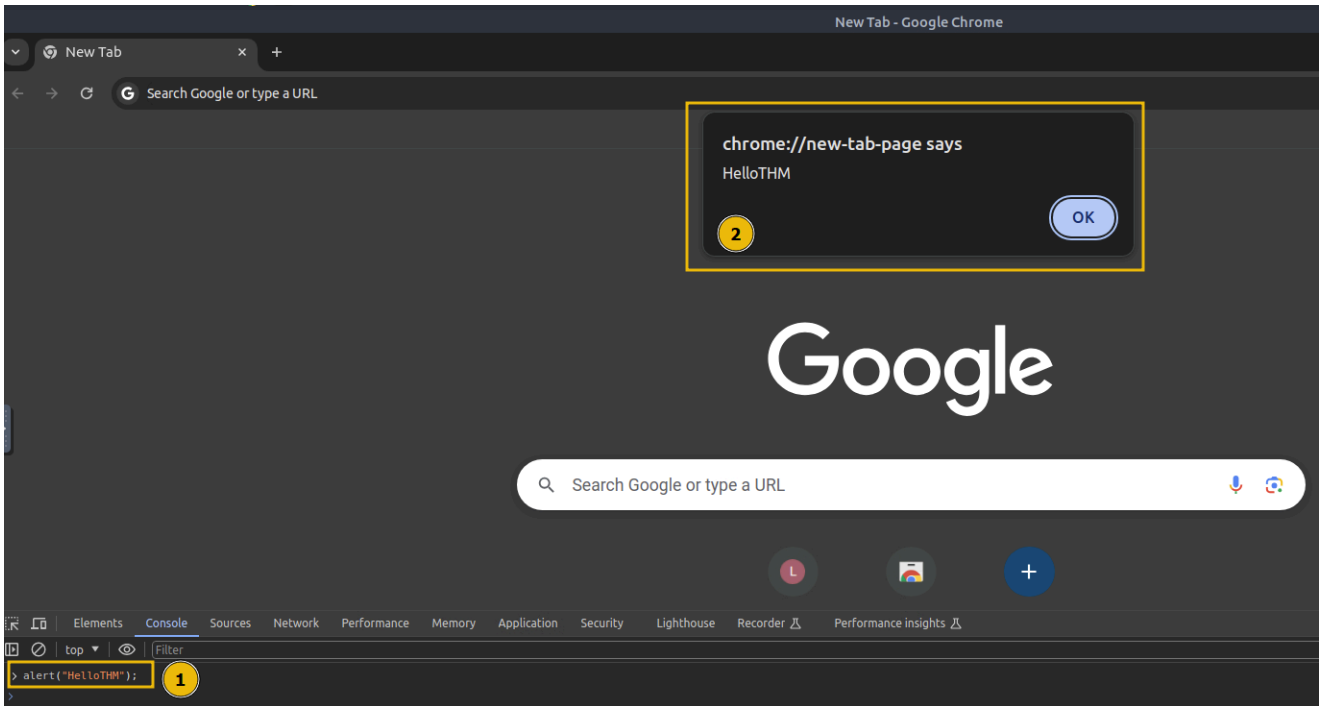
1. Tarayıcıda sayfaya sağ tıkla -> **Sayfa Kaynağını Görüntüle (View Page Source)**.
2. CTRL + F ile `<script` kelimesini ara.
3. Eğer `src` parametresi yoksa ve kodlar doğrudan sayfadaysa bu **Internal JS**'dir.
4. Eğer `src="/js/app.js"` gibi bir yol varsa bu **External JS**'dir.

Pentester İpucu: Bazen harici JS dosyaları bulut sunucularda (CDN) veya farklı domainlerde tutulur. Bu dosyaların içeriğini inceleyerek, uygulamanın kullandığı kütüphanelerin versiyonlarını (jQuery, React vb.) öğrenebilir ve bilinen zafiyetleri (CVE) araştırabilirsin.

Diialog Fonksiyonlarının Kötüye Kullanımı (Abusing Dialogue Functions)

JavaScript'in temel amaçlarından biri, kullanıcıyla etkileşime girmek için diyalog kutuları sunmak ve sayfa içeriğini dinamik olarak güncellemektir. JS; **alert**, **prompt** ve **confirm** gibi yerleşik fonksiyonlarla bu etkileşimi sağlar. Ancak bu fonksiyonlar güvenli bir şekilde yapılandırılmazsa, saldırganlar tarafından **XSS (Cross-Site Scripting)** gibi saldırıları gerçekleştirmek için birer araç olarak kullanılabilir.

1. Alert (Uyarı) Fonksiyonu



`alert()` fonksiyonu, kullanıcıya üzerinde sadece bir "Tamam" (OK) butonu bulunan küçük bir pencere gösterir. Genellikle bilgilendirme veya uyarı amaçlı kullanılır.

- **Kullanımı:** `alert("Mesajınız");`
- **Siber Güvenlikteki Önemi:** Bir siber güvenlik araştırmacısı için `alert()` fonksiyonu, bir web sitesinde **XSS zafiyeti** olup olmadığını kanıtlamak için kullanılan en yaygın "PoC" (Proof of Concept - Kavram Kanıtı) aracıdır. Eğer bir input alanına yazdığınız `<script>alert(1)</script>` kodu ekranda bir uyarı kutusu çıkarıyorsa, o site JavaScript enjeksiyonuna karşı savunmasız demektir.

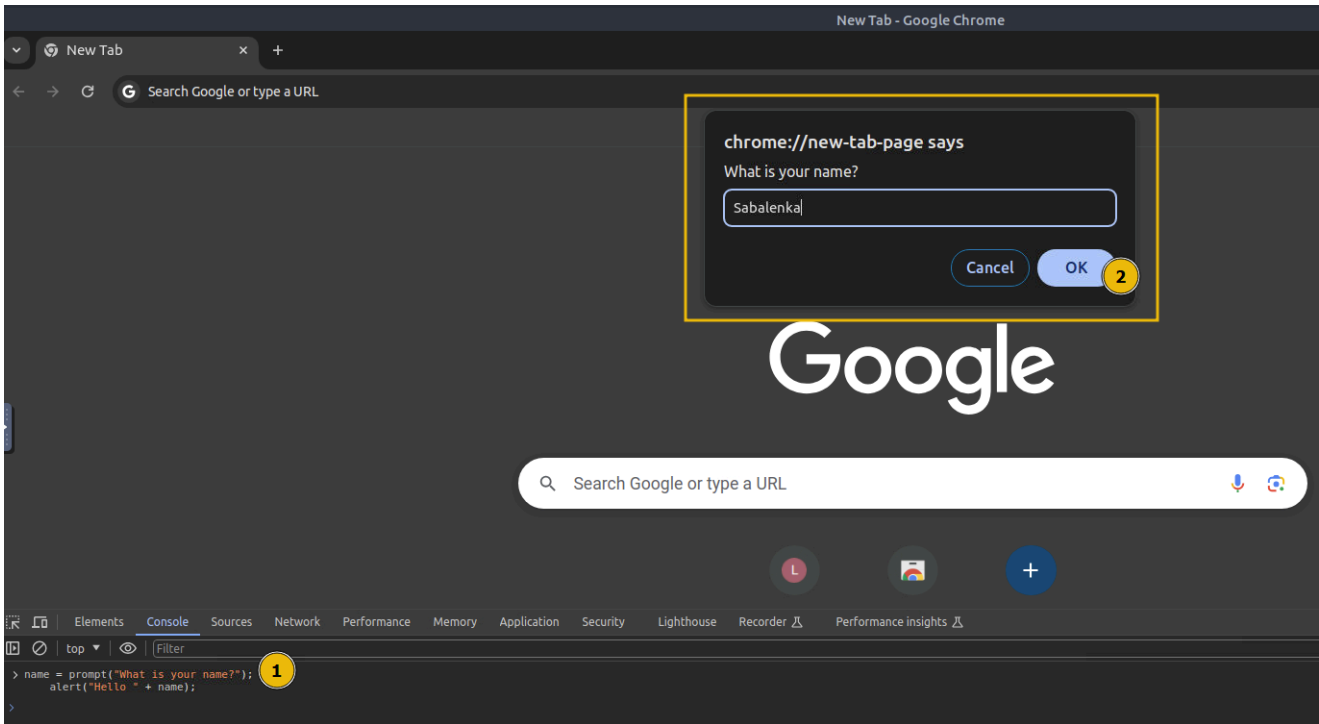
Deneyelim: Chrome konsolunu açıp şunu yazın:

JavaScript

```
alert("Hello THM");
```

Ekranda "Hello THM" yazan bir kutucuk belirecektir.

2. Prompt (Giriş İsteme) Fonksiyonu



`prompt()` fonksiyonu, kullanıcıdan veri girişi bekleyen bir diyalog kutusu açar. Kullanıcı "Tamam" derse girilen değeri döndürür, "İptal" derse `null` değerini döndürür.

- **Kullanımı:** `prompt("Sorunuz?");`
- **Siber Güvenlikteki Önemi:** Saldırganlar bu kutuyu kullanarak kullanıcıyı kandırabilir. Örneğin, sahte bir "Oturum süreniz doldu, lütfen şifrenizi girin" kutusu çıkararak kullanıcı bilgilerini çalabilirler.

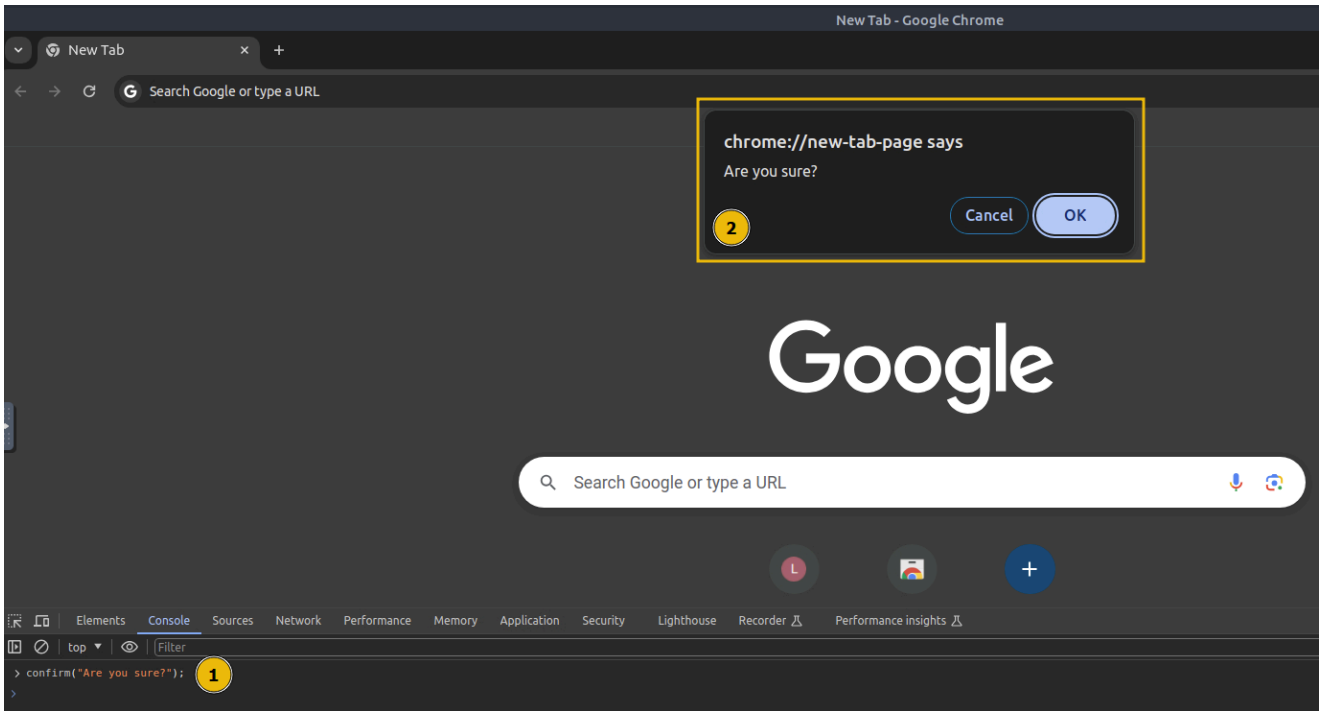
Konsol Testi: Kullanıcıdan isim alıp onu selamlayan şu kodu konsola yapıştırın:

JavaScript

```
// Kullanıcıdan isim al ve 'name' değişkenine ata
let name = prompt("Adınız nedir?");

// Alınan ismi alert ile ekrana bas
alert("Merhaba " + name);
```

3. Confirm (Onaylama) Fonksiyonu



`confirm()` fonksiyonu, kullanıcıya "Tamam" ve "İptal" seçeneklerini sunar. Eğer "Tamam" tıklanırsa `true` (doğru), "İptal" tıklanırsa `false` (yanlış) değerini döndürür.

- **Kullanımı:** `confirm("Emin misiniz?");`
- **Mantık:** Bu fonksiyon genellikle kritik işlemlerden (silme, gönderme vb.) önce son bir kontrol olarak kullanılır.

Deneyelim:

JavaScript

```
let onay = confirm("Devam etmek istiyor musunuz?");
console.log(onay); // Tıkladığınız butona göre true veya false basar.
```

4. Saldırganlar Bu Fonksiyonları Nasıl Suistimal Eder?

Basit bir diyalog kutusu zararsız görünebilir, ancak bir saldırgan bunu kullanıcının tarayıcısını kilitlemek veya sinir bozucu bir deneyim yaşatmak için kullanabilir.

Örnek Senaryo: Sonsuz Döngü / Spam Alert

Size bir e-posta ile `fatura.html` dosyası geldiğini düşünün. İçinde şu kodlar olabilir:

HTML

```
<!DOCTYPE html>
<html lang="tr">
<head>
  <title>Hacked</title>
</head>
<body>
```

```
<script>
  // Döngü 3 kez döner ve 3 kez alert çıkarır.
  for (let i = 0; i < 3; i++) {
    alert("Hacked!");
  }
</script>
</body>
</html>
```

Analiz:

- Yukarıdaki kodda döngü sayısı 3 yerine 500 veya sonsuz bir döngü yapılıyorsa, kullanıcı tarayıcıyı kapatana kadar (veya tarayıcı sekmesi çökene kadar) yüzlerce kutucuğu tek tek kapatmak zorunda kalırdı.
- Bu, en temel düzeyde bir **Denial of Service (Servis Dışı Bırakma)** saldırısı mantığıdır; kullanıcının tarayıcıyı normal şekilde kullanmasını engeller.

Kritik Not: Asla güvenmediğiniz kaynaklardan gelen HTML dosyalarını açmayın veya bilmediğiniz sitelerden gelen JavaScript dosyalarının çalışmasına izin vermeyin. Modern tarayıcılar artık "Bu sitenin daha fazla diyalog oluşturmasını engelle" gibi seçenekler sunsa da, bu fonksiyonlar hala sosyal mühendislik saldırılarında aktif olarak kullanılmaktadır.

Kontrol Akış Yapılarını Atlatma (Bypassing Control Flow Statements)

Bu bölümde, JavaScript'teki **if-else** gibi karar mekanizmalarının mantığını ve en önemlisi, bu mekanizmaların istemci tarafında (Client-Side) çalışmasının siber güvenlik açısından yarattığı devasa riskleri inceleyeceğiz. Bir geliştirici, kritik bir kararı (örneğin giriş izni) sadece JS ile veriyorsa, bu durum "atlatılmaya" (bypass) davetiye çıkarır.

1. Karar Mekanizmaları: if-else Mantığı

JavaScript'te **Control Flow** (Kontrol Akışı), kodun hangi sırayla ve hangi şartlar altında çalışacağını belirler. En temel yapı **if-else** bloğudur.

- if:** Belirtilen koşul **true** (doğru) ise içindeki kod bloğunu çalıştırır.
- else:** Koşul **false** (yanlış) ise alternatif kod bloğunu çalıştırır.

Uygulama: Yaş Doğrulama (age.html)

Aşağıdaki kod, kullanıcının girdiği yaşa göre sayfadaki metni dinamik olarak değiştirir:

HTML

```
<!DOCTYPE html>
<html lang="tr">
```

```
<head>
  <title>Yaş Doğrulama</title>
</head>
<body>
  <h1>Yaş Doğrulama Sistemi</h1>
  <p id="message"></p>

  <script>
    // Kullanıcıdan veri al
    let age = prompt("Kaç yaşındasınız?");

    // Karar yapısı
    if (age >= 18) {
      // Koşul sağlanırsa (18 ve üstü)
      document.getElementById("message").innerHTML = "Yetişkinsiniz,
giriş yapabilirsiniz.";
    } else {
      // Koşul sağlanmazsa (17 ve altı)
      document.getElementById("message").innerHTML = "Reşit
değilsiniz, erişim engellendi.";
    }
  </script>
</body>
</html>
```

Analiz: Bu kodda `age >= 18` ifadesi bir **mantıksal denetimdir**. Kullanıcı "20" girerse ifade `true` döner ve ilk blok çalışır. Ancak bir pentester olarak şunu bilmeliyiz: **Bu kontrol sadece kullanıcının tarayıcısında gerçekleşiyor.**

2. İstemci Tarafli Giriş Formlarını Atlatma (Bypassing Login Forms)

Bazı geliştiriciler, kullanıcı adı ve şifre kontrolünü sunucu tarafında (PHP, Python, Node.js vb.) yapmak yerine, hatalı bir şekilde sadece JavaScript ile tarayıcıda yaparlar. Bu, siber güvenlikteki en büyük "fail" senaryolarından biridir.

Senaryo: `login.html` İncelemesi

Diyelim ki bir sistem, sadece kullanıcı adı "admin" olanlara izin veriyor. Kod muhtemelen şuna benziyor:

JavaScript

```
let username = prompt("Kullanıcı Adı:");
let password = prompt("Şifre:");

if (username === "admin" && password === "p@ss123") {
  alert("Giriş Başarılı! Hoş geldin admin.");
}
```

```
// Gizli içeriği gösteren fonksiyonu çağır
showSecretContent();
} else {
    alert("Hatalı bilgiler!");
}
```

Neden Güvensiz? (Pentester Gözüyle)

Bu yöntem neden bir "güvenlik açığıdır"?

1. **Kodun Görünürlüğü:** Sayfa kaynağına (View Source) baktığımızda, geçerli kullanıcı adının "admin" ve şifrenin "p@ss123" olduğunu doğrudan görebiliriz.
2. **Mantığı Değiştirme:** Tarayıcı konsolunu (F12) açarak `if` koşulunu tamamen devre dışı bırakabilir veya `username` değişkenine zorla "admin" değerini atayabiliriz.
3. **Fonksiyonu Doğrudan Çağırma:** Eğer giriş başarılı olduğunda çalışan bir fonksiyon varsa (`showSecretContent()` gibi), biz `if` kontrolüyle hiç uğraşmadan konsola doğrudan bu fonksiyonun adını yazıp `Enter` a basarak içeriğe erişebiliriz.

Kritik Not: İstemci vs. Sunucu Kontrolü

- **İstemci Taraflı (JS):** Sadece kullanıcı deneyimi içindir (örneğin şifre boşsa uyarı vermek). Asla **güvenlik kararı** (yetki verme) için kullanılmamalıdır.
- **Sunucu Taraflı:** Gerçek güvenlik burada başlar. Şifre kontrolü sunucuda, veritabanı sorgusuyla yapılmalıdır. Çünkü saldırgan sunucudaki `if` bloğuna müdahale edemez.

Minified ve Obfuscated (Karmaşılaştırılmış) Dosyaları İnceleme

Şu ana kadar JavaScript kodlarını birer açık kitap gibi okuduk. Ancak gerçek dünyada ve profesyonel web uygulamalarında kodlar genellikle "insan tarafından okunamaz" bir halde karşımıza çıkar. Bir siber güvenlik araştırmacısı olarak bu "anlaşılmaz" yığınin arkasındaki mantığı nasıl çözeceğimizi bilmemiz gerekir.

1. Minification (Küçültme) Nedir?

Minification, bir JS dosyasının işlevselliğini bozmadan boyutunu küçültme işlemidir.

- **Neler yapılır?:** Gereksiz boşluklar, satır başları ve yorum satırları silinir. Değişken isimleri `kullaniciAdi` yerine `a` gibi tek harfe indirgenir.
- **Amaç:** Sayfa yükleme hızını artırmak ve bant genişliğinden tasarruf etmektir.
- **Etki:** Tarayıcı bu kodu mükemmel çalıştırır ancak bir insan için okumak tam bir işkencedir.

2. Obfuscation (Karmaşılaştırma) Nedir?

Obfuscation, kodun sadece boyutunu küçültmekle kalmaz, aynı zamanda tersine mühendisliği (reverse engineering) zorlaştırmak için kasıtlı olarak karmaşık hale getirir.

- **Neler yapılır?:** Mantıksal akış karıştırılır, anlamsız fonksiyon isimleri eklenir ve veriler (stringler) hex/base64 gibi formatlara dönüştürülür.
- **Güvenlik Notu:** Zararlı yazılım (malware) geliştiricileri, antivirüslerin ve analistlerin kodu anlamasını zorlaştırmak için bu yöntemi çok sık kullanır.

3. Uygulamalı Örnek: `hello.js`

Önce temiz kodun nasıl görüldüğüne bakalım:

Dosya: `hello.js`

JavaScript

```
function hi() {  
    alert("Welcome to THM");  
}  
hi();
```

Bu kodu tarayıcıda **Inspect (İncele) > Sources** sekmesi altında gördüğümüzde, her şey net ve anlaşılırdır.

4. Obfuscation İşlemi ve Analizi

Yukarıdaki temiz kodu bir "obfuscator" aracına koyduğumuzda, karşımıza çıkan çıktı şu devasa ve anlamsız yapıya dönüşür:

JavaScript

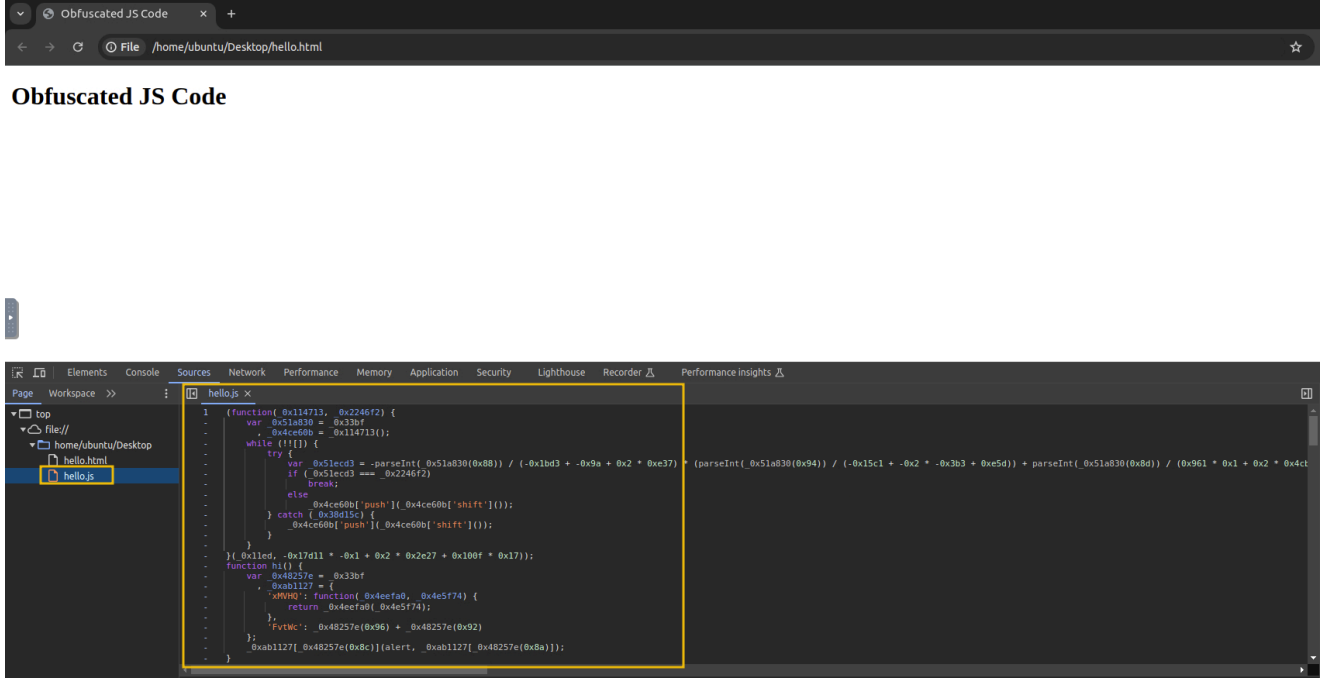
```
// Örnek Obfuscated Kod Parçası  
(function(_0x114713,_0x2246f2){var _0x51a830=_0x33bf ... // uzayıp giden  
anlamsız karakterler
```

Bu kod neden hala çalışıyor?

Tarayıcı için `function hi()` demekle `function _0x33bf()` demek arasında bir fark yoktur. Tarayıcı sadece referanslara bakar. Kodun içindeki karmaşık matematiksel işlemler (`-0x1bd3+-0x9a+0x2*0xe37`) aslında basit bir sayıyı veya karakteri temsil eder. Tarayıcı bu işlemleri anında yapar ve orijinal `alert("Welcome to THM")` komutuna ulaşır.

Pentest İpucu: Bir sitede bu tarz "garip" bir JS dosyası gördüğünde, içinde saklanan gizli bir

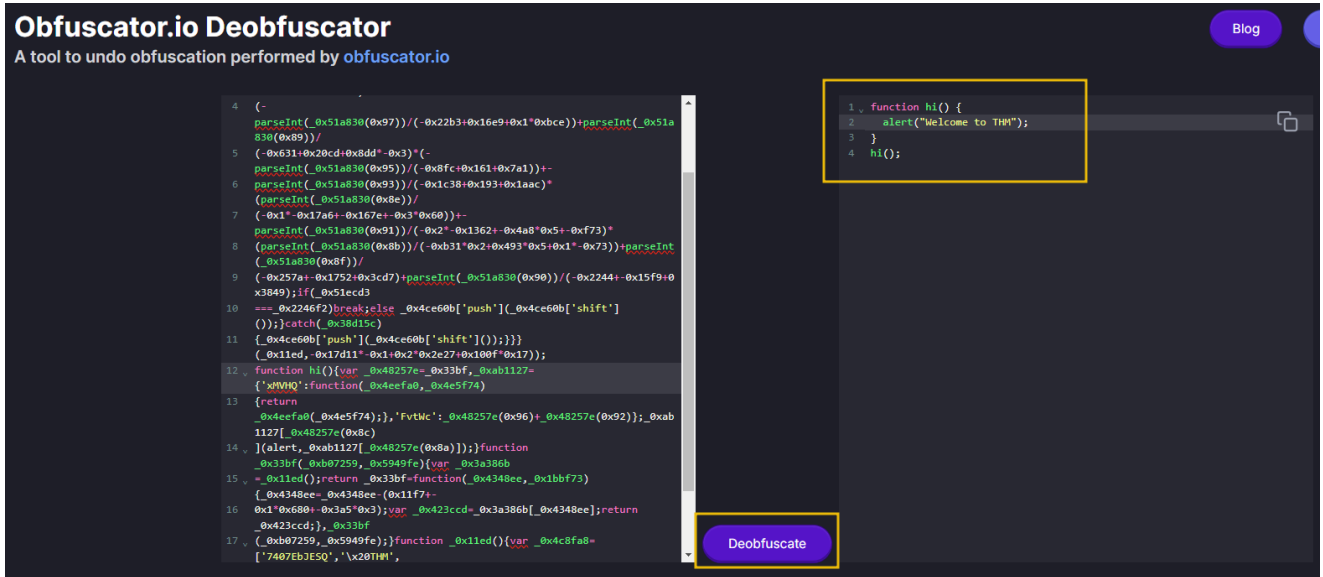
API anahtarı, şifreleme anahtarı veya gizli bir uç nokta (endpoint) olabilir.



5. Deobfuscation: Kodu Eski Haline Getirme

Karmaşılaştırılmış bir kodu analiz etmek için saatlerce matematik yapmamıza gerek yok. Bunun için **Deobfuscator** veya **Beautifier** araçlarını kullanırız.

- **İşlem:** Karmaşık kodu kopyalayıp deobfuscate aracına yapıştırdığımızda, araç karmaşık değişkenleri temizler ve kodun okunabilir hiyerarşisini geri getirir.
- **Sonuç:** Orijinal koda %100 aynı olmasa bile (çünkü orijinal değişken isimleri kaybolmuş olabilir), kodun **ne yaptığını** anlayabileceğimiz temiz bir çıktı elde ederiz.



Özet Tablo: Hangisi Hangisi?

Özellik	Minification	Obfuscation
Temel Amaç	Performans / Hız	Gizlilik / Zorlaştırma

Özellik	Minification	Obfuscation
Okunabilirlik	Zor (Boşluk yok)	Çok Zor (Mantık karmaşık)
Dosya Boyutu	Küçülür	Genellikle Büyür
Tersine Çevirme	"Pretty Print" ile kolayca çözülür	Deobfuscator araçları gerektirir

JavaScript Güvenliği: En İyi Uygulamalar (Best Practices)

Bir web uygulaması geliştirirken veya bir uygulamayı siber güvenlik açısından test ederken (pentest), JavaScript'in nasıl "suistimal edilebileceğini" bilmek kadar, bu saldırı yüzeyini nasıl daraltacağımızı da bilmemiz gerekir. Aşağıdaki maddeler, hem geliştiriciler hem de güvenlikçiler için altın kurallardır.

1. Sadece İstemci Taraflı Doğrulamaya Güvenmeyin

Client-Side Validation (İstemci Taraflı Doğrulama), kullanıcı bir formu doldururken (örneğin e-posta formatı doğru mu, şifre yeterince uzun mu) anlık geri bildirim vermek için harikadır. Ancak bu, **asla tek başına bir güvenlik önlemi değildir**.

- Risk:** Kullanıcı tarayıcısında JavaScript'i tamamen devre dışı bırakabilir veya konsol üzerinden doğrulama fonksiyonlarını (`if` bloklarını) manipüle ederek sunucuya istediği hatalı veriyi gönderebilir.
- Çözüm:** Doğrulama her zaman hem istemcide (kullanıcı deneyimi için) hem de **sunucu tarafında** (gerçek güvenlik için) yapılmalıdır. Sunucu, gelen her veriyi "kirli" kabul etmeli ve tekrar kontrol etmelidir.

2. Güvenilmeyen Kütüphaneleri Dahil Etmekten Kaçın

JavaScript'te `<script src="...">` etiketiyle dışarıdan kütüphane çağırmak çok kolaydır. Ancak bu kütüphanenin kaynağı hayati önem taşır.

- Risk:** Saldırganlar, popüler kütüphanelerin isimlerine çok benzer (Typosquatting) zararlı paketler oluşturup internete yüklerler. Eğer yanlışlıkla bu zararlı kütüphaneyi projenize eklerseniz, web uygulamanızın kontrolünü saldırgana teslim etmiş olursunuz.
- Çözüm:** Sadece resmi ve güvenilir kaynaklardan (resmi CDN'ler, güvenilir NPM paketleri vb.) kütüphane çekin. Mümkünse kütüphaneyi kendi sunucunuza indirip oradan çağırın.

3. Hardcoded (Sabit Kodlanmış) Sırlardan Kaçın

JavaScript kodunun içine asla **API anahtarları**, **erişim tokenları** veya **kullanıcı şifreleri** yazılmamalıdır.

Neden? Çünkü daha önce gördüğümüz gibi, herhangi bir kullanıcı "Sayfa Kaynağını Görüntüle" diyerek veya "Sources" sekmesine bakarak bu bilgileri saniyeler içinde

çalabilir.

JavaScript

```
// KÖTÜ UYGULAMA (Asla yapma!)  
const privateAPIKey = 'pk_TryHackMe-1337';  
const adminPassword = 'admin_sifresi_burada';
```

- **Çözüm:** Bu tür hassas veriler sunucu tarafındaki çevre değişkenlerinde (`environment variables`) tutulmalı ve sadece ihtiyaç duyulduğunda güvenli API çağrılarıyla işlenmelidir.

4. Minification ve Obfuscation Kullanın

Kodun boyutunu küçültmek (**Minify**) ve mantığını karmaşıktırmak (**Obfuscate**), kodu üretim (production) ortamına alırken standart bir adımdır.

- **Fayda:** Sayfanın hızlı yüklenmesini sağlar ve saldırganın kodun mantığını çözmesini zorlaştırır.
- **Gerçekçi Yaklaşım:** Unutmayın, bu yöntemler kodu "kırılamaz" yapmaz; sadece saldırganın işini zorlaştırır ve zaman kaybettirir. Kararlı bir saldırgan eninde sonunda kodu deobfuscate ederek analiz edebilir (Reverse Engineering).

Özet Güvenlik Kontrol Listesi

Kural	Neden Önemli?	Temel Mantık
Çift Taraflı Kontrol	JS atlatılabilir.	Sunucu tarafı (Back-end) kontrolü şarttır.
Kaynak Kontrolü	Zararlı kod sızabilir.	Sadece bilinen/resmi kütüphaneleri kullan.
Gizli Veri Yok	Kod herkes tarafından okunabilir.	API anahtarlarını kodun içine gömme.
Kod Karmaşıklığı	Analizi zorlaştırır.	Minify ve Obfuscate işlemlerini uygula.